

ระบบวางแผนและจัดตารางเรียน มทร.ธัญบุรี

ระบบช่วยนักศึกษาวิศวกรรมคอมพิวเตอร์จัดตารางเรียนแต่ละเทอม แล้วตรวจให้
ก่อนวันลงทะเบียนจริงว่า เวลาเรียนชน เวลาสอบชน หน่วยกิตเกิน วิชาบังคับก่อน
ยังไม่ผ่าน หรือที่นั่งเต็ม หรือเปล่า

ดาวน์โหลดเป็น PDF (10 หน้า) →

9 โมดูล	6 กฎที่ต้องผ่าน (C1-C6)	5 คำเตือน (W1-W5)	9-21 หน่วยกิตต่อเทอม	182 บิตในกริดเวลา
---------	-------------------------	-------------------	----------------------	-------------------

01 · เริ่มจากตรงนี้

ภาพรวมใน 1 นาที

ระบบแบ่งเป็น 4 ชั้น เหมือนสำนักงานที่มีหลายแผนก นักศึกษาคุยกับชั้นหน้าบ้านอย่าง
เดียว ไม่ต้องรู้ว่าข้างในมีกี่แผนก



ลูกศรคือทิศทางที่คำขอวิ่ง — จากบนลงล่าง แล้วคำตอบวิ่งกลับขึ้นมาทางเดิม

9 โมดูล ทำอะไรบ้าง

แต่ละโมดูลคือ 1 โฟลเดอร์ใน repo และมีเจ้าของคนเดียว ทำงานแยกกันได้โดยไม่ชนกัน กล่องสี่เหลี่ยมคือคำเปรียบเทียบให้เห็นภาพเร็ว ๆ

ชั้น 1 หน้าบ้าน		สิ่งที่นักศึกษาเห็นและกด
01_web_app :3001 รวมเข้า main แล้ว	หน้าเว็บสำหรับนักศึกษา แสดงตารางเรียน ปุ่มกด ช่องค้นหา และกล่องแจ้งว่าตารางมีปัญหาอะไร เปรียบเทียบ เคาเตอร์หน้าร้าน — เป็นที่เดียวที่ลูกค้าเดินมาคุย ไม่ต้องรู้ว่าหลังร้านมีกี่แผนก การกดของผู้ใช้ → เรียก 02 → วาดตารางออกมา เจ้าของ เฟส (First) · อยู่บน main แล้ว (30 ไฟล์)	
09_main_app Vercel / Pages ใช้งานได้จริง	แอปตัวจริงที่รวมทุกอย่างไว้ที่เดียว ยกตรรกะสำคัญจากโมดูล 06 และ 07 มารวมไว้ ทำให้ deploy ขึ้นเว็บได้โดยไม่ต้องรัน container ทั้ง 8 ตัว เปรียบเทียบ ร้านสาขาย่อยที่มีทุกอย่างครบในตึกเดียว — เล็กกว่า แต่เปิดให้คนใช้จริงได้เลยวันนี้ Next.js + Python serverless → เว็บที่เปิดใช้ได้ เจ้าของ เฟส (First) · อยู่บน main แล้ว	

02_api_backend
ประตูเดียวที่หน้าบ้านคุยด้วย

:8000

รวมเข้า main แล้ว

ประตูเดียวที่หน้าบ้านคุยด้วย
 รับคำขอทุกอย่างจากหน้าเว็บ ตรวจสอบว่าส่งมาถูกรูปแบบไหม แล้วกระจายไปยังโมดูลที่เกี่ยวข้อง

เปรียบเทียบ
 เจ้าหน้าที่ประชาสัมพันธ์ — ฟังเรื่องแล้วบอกว่า “เรื่องนี้ต้องไปห้องนั้น” แทนที่จะให้เราเดินหาเอง

ค่าขอจาก 01 → 04/05/06/07/08 → รวมค่าตอบส่งกลับ

เจ้าของ fffalafair · อยู่บน main แล้ว (100 ไฟล์)

03_ai_router_agent
นักตัดสินใจว่าคำถามนี้ต้องส่งให้ใคร

:8100

รวมเข้า main แล้ว

นักตัดสินใจว่าคำถามนี้ต้องส่งให้ใคร
 นักศึกษาพิมพ์เป็นภาษาคน ตัวนี้อ่านแล้วแยกว่าเป็นเรื่องจัดตาราง เรื่องคันวิชา หรือเรื่องระเบียบ

เปรียบเทียบ
 ลามที่ฟังคำถามกำกวมแล้วรู้ว่าต้องพาไปหาใคร

ข้อความจากผู้ใช้ → แยกเจตนา → ส่งให้ 06 หรือ 07

เจ้าของ Automatic28m (Phanlop) · อยู่บน main แล้ว (10 ไฟล์)

06_schedule_
 conflict_engine

:8600

เสร็จแล้ว

หัวใจของระบบ — ตรวจสอบตารางชนและจัดตารางให้

ตรวจสอบ 6 ข้อที่ทำให้ลงทะเบียนไม่ผ่าน (C1-C6) และ 5 ข้อที่ควรรู้ไว้ (W1-W5) แล้วยังจัดตารางที่ไม่ชนให้อัตโนมัติได้ด้วย

เปรียบเทียบ
 รุ่นที่ที่จัดตารางทั้งคณะได้ กวาดตาดูแป๊บเดียวก็บอกได้ว่า “ตัวนี้ชนกับตัวนั้นวันพุธบ่าย”

รายการหมู่เรียนที่เลือก → C1-C6 / W1-W5 → พร้อมวิธีแก้

เจ้าของ **cheewakorn** · อยู่บน main แล้ว (25 ไฟล์ ใช้ CP-SAT / OR-Tools)

07_rag_llm_engine

:8700

เสร็จแล้ว

ตอบคำถามเรื่องระเบียบ พร้อมบอกว่าอ้างจากไหน

ค้นเอกสารระเบียบก่อน แล้วค่อยเรียบเรียงคำตอบจากสิ่งที่ค้นเจอ ถ้าไม่มีข้อมูลเรื่องนั้นจะบอกตรง ๆ ว่าไม่รู้ไม่เข้าใจ

เปรียบเทียบ
 เจ้าหน้าที่ที่เปิดเล่มระเบียบให้ดูทีละหน้า แทนที่จะตอบจากความจำ

คำถามภาษาไทย → ค้นเอกสาร → คำตอบ + แหล่งอ้างอิง

เจ้าของ **เพ็ญ (First)** · อยู่บน main แล้ว (105 เทสผ่าน)

08_recommendation_
 _feedback

:8800

รวมเข้า main แล้ว

ทัศนคติที่ว่าระบบช่วยได้จริงไหม

บันทึกว่าคนถามอะไรบ่อย ระบบตอบผิดตรงไหน แผนที่จะจัดให้ถูกนำไปใช้จริงกี่เปอร์เซ็นต์

เปรียบเทียบ
 กล่องรับความคิดเห็นหน้าร้าน ที่เอามาปรับปรุงรอบหน้า

การใช้งานจริง → สถิติ → ข้อเสนอปรับปรุง

เจ้าของ **marisa46054 (Marisa)** · อยู่บน main แล้ว (23 ไฟล์)

04_course_data_services

:8400

ยังไม่มีเจ้าของ

ไปถึงตารางสอนจากระบบทะเบียนมา

อ่านหน้าเว็บของระบบทะเบียน แล้วแกะออกมาเป็นข้อมูลที่โปรแกรมใช้ได้ — รายวิชา หมู่เรียน วัน-เวลา ที่นั่ง

เปรียบเทียบ คนที่ไปถ่ายเอกสารตารางสอนจากบอร์ดหน้าคณะมาให้

หน้าเว็บ oreg3 → แกะข้อมูล → JSON

ยังไม่มีเจ้าของ · มี parser อยู่บน main แล้ว

05_data_integrat

:8500

รวมเข้า main แล้ว

นำข้อมูลให้สะอาดและเชื่อมโยงกัน

ชื่อวิชาที่เขียนไม่เหมือนกันให้เป็นแบบเดียวกัน แล้วต่อเส้นว่าวิชาไหนต้องเรียนก่อนวิชาไหน

เปรียบเทียบ บรรณารักษ์ที่จัดหนังสือเข้าชั้นให้ถูกหมวด และติดป้ายว่าเล่มนี้ต้องอ่านเล่มไหนก่อน

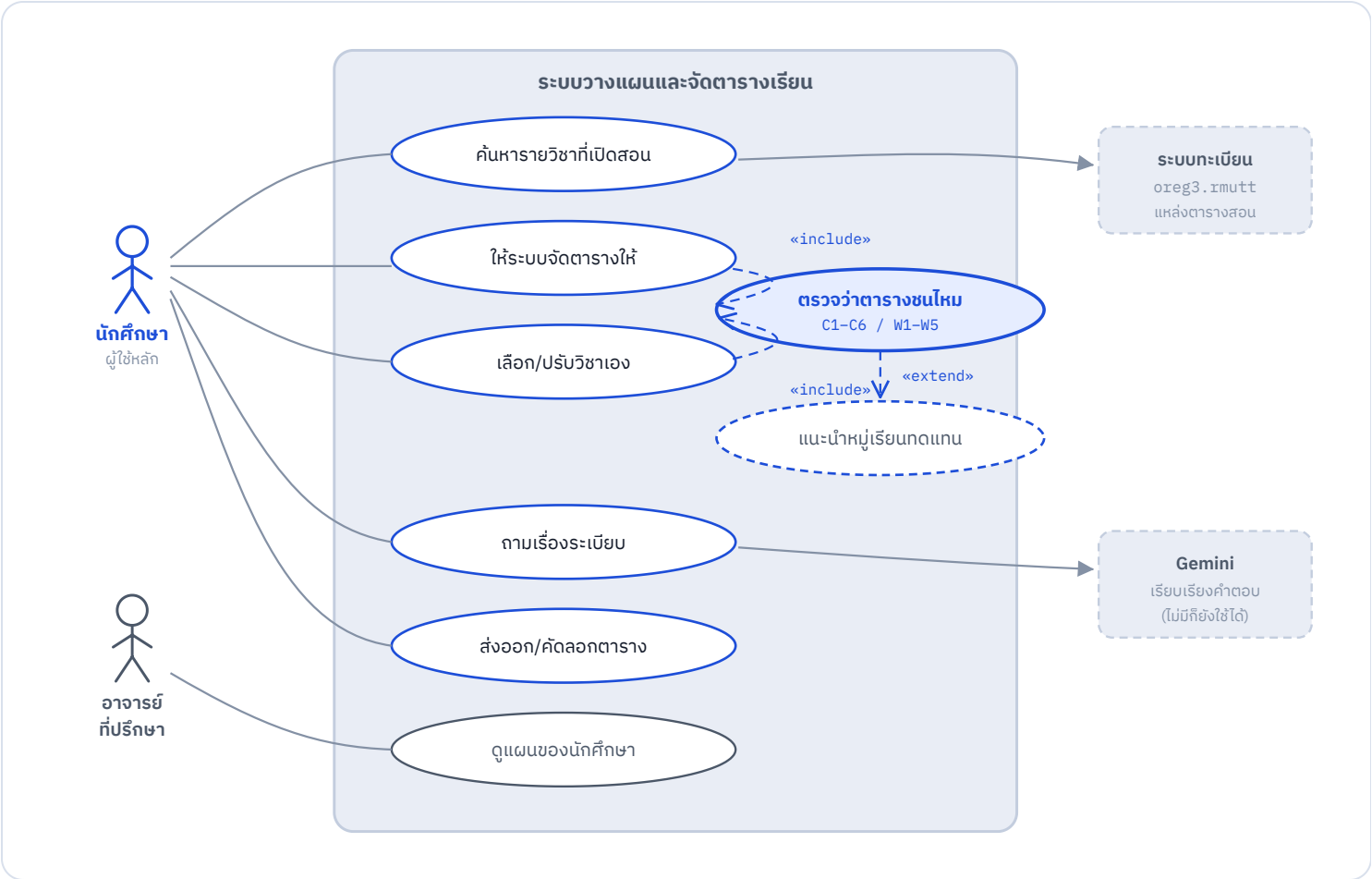
ข้อมูลดิบจาก 04 → ทำให้เป็นมาตรฐาน → ผังวิชาบังคับก่อน

เจ้าของ DevnameJay · อยู่บน main แล้ว (14 ไฟล์)

03 · ใครใช้ทำอะไรได้บ้าง

Use Case Diagram

คนตัวเล็ก ๆ คือ ผู้ใช้ วงรีคือ สิ่งที่ได้ทำ กรอบใหญ่คือขอบเขตของระบบ กล้องเส้นประด้านขวาคือระบบภายนอกที่เราไปขอข้อมูลมาใช้

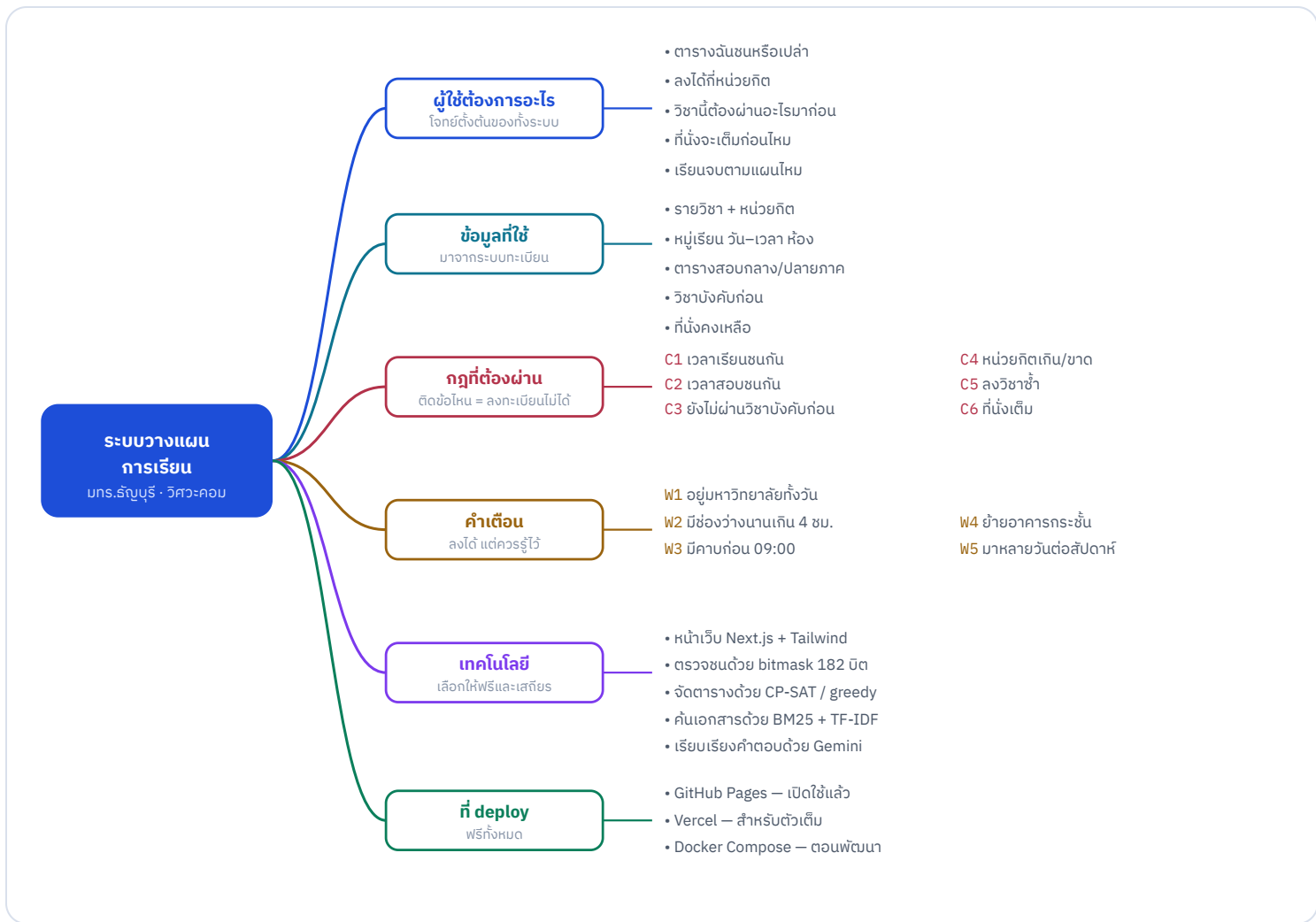


สัญลักษณ์	อ่านว่า	ตัวอย่างในระบบนี้
คนตัวเล็ก	ผู้ใช้ หรือระบบที่มาใช้งานเรา	นักศึกษา, อาจารย์ที่ปรึกษา
วงรี	สิ่งที่ทำได้ 1 อย่าง	“ให้ระบบจัดตารางให้”
เส้นทึบ	ผู้ใช้คนนี้ใช้ความสามารถนี้ได้	นักศึกษา → ค้นหาวิชา
«include»	ทำอันนี้ ต้อง ทำอันนั้นด้วยเสมอ	จัดตารางให้ → ต้องตรวจว่าชนไหมทุกครั้ง
«extend»	ทำเพิ่มให้ เฉพาะบางกรณี	เจอว่าชน → ถึงค่อยแนะนำหมู่ทดแทน
กล่องเส้นประ	ระบบภายนอกที่เราไม่ได้เขียนเอง	ระบบทะเบียน, Gemini

04 · มงภาพรวมทีเดียว

มายด์แมปทั้งระบบ

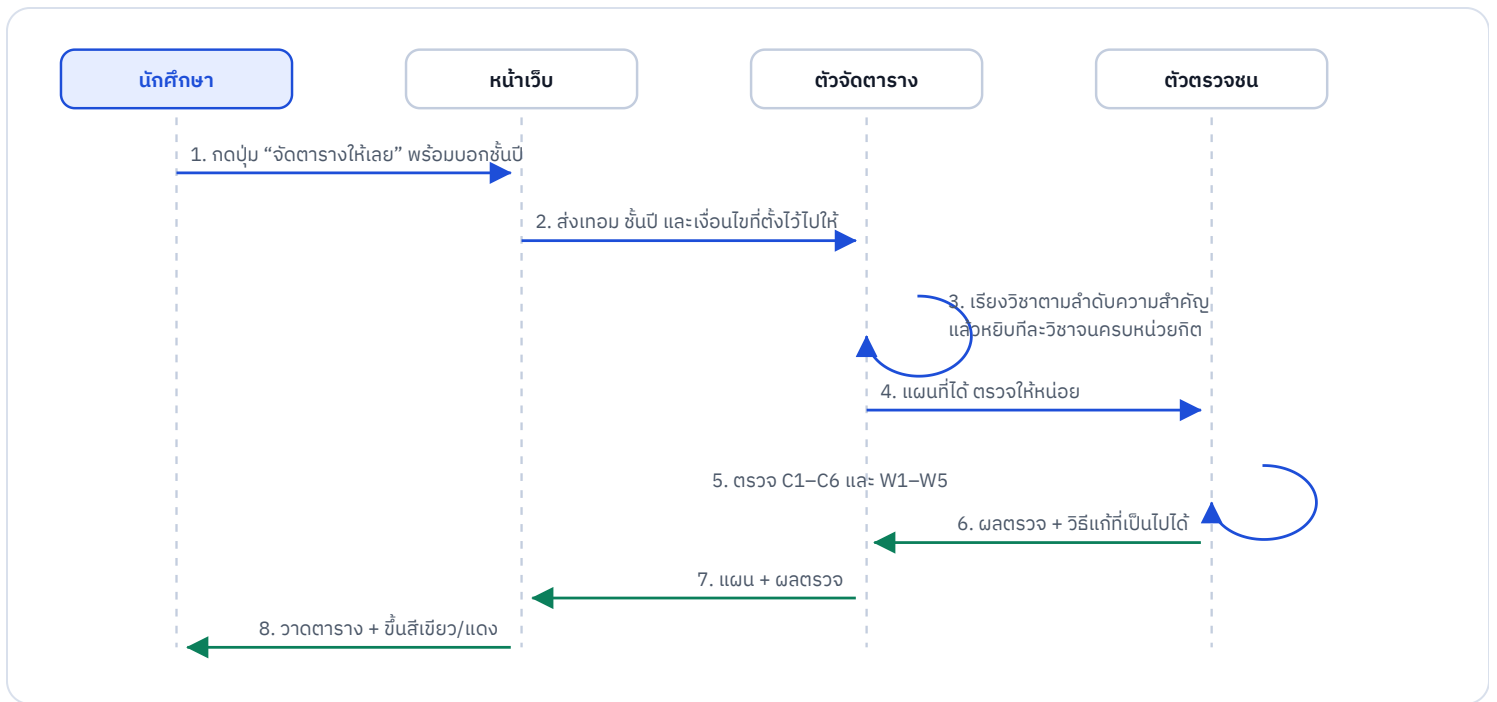
อ่านจากกลางออกไปขวา — เริ่มที่ ผู้ใช้ต้องการอะไร แล้วไล่ดูว่าระบบใช้อะไรตอบ
โจทย์นั้น



05 · ทำงานยังงี้

ขั้นตอนการทำงาน ที่ละขั้น

ยกตัวอย่างที่ใช้บ่อยที่สุด — นักศึกษากดปุ่ม “จัดตารางให้เลย” แล้วเกิดอะไรขึ้นบ้าง



อธิบายทีละขั้น

- นักศึกษาบอกว่าตัวเองอยู่ปีไหน**
เลือกภาคการศึกษากับชั้นปี แค่นี้พอ ไม่ต้องกรอกรหัสนักศึกษาหรือสมัครสมาชิก
- ระบบเดาว่าผ่านวิชาปีก่อนหน้ามาแล้ว**
ถ้าเลือกปี 3 ระบบจะถือว่าผ่านวิชาปี 1-2 มาแล้ว เพื่อปลดล็อกวิชาที่มีเงื่อนไขบังคับก่อน — เป็นการเดา ปิดเองได้ในเงื่อนไข
- เรียงลำดับว่าวิชาไหนควรได้ลงก่อน**
วิชาของชั้นปีตัวเองมาก่อน ตามด้วยวิชาซีฟเลือก แล้วค่อยวิชาปีอื่น
- หยิบทีละวิชา เช็กทุกครั้งว่าชนของเดิมไหม**
ทั้งเวลาเรียนและเวลาสอบ ถ้าชนก็ลองหุ่อื่น ถ้าทุกหุ่ชนหมดก็ข้ามวิชานั้นแล้วบันทึกเหตุผลไว้บอกผู้ใช้
เทียบเวลาด้วย bitmask 182 บิต (7 วัน × 26 ช่วงครึ่งชั่วโมง 08:00-21:00)
- หยุดเมื่อครบเพดานหน่วยกิต**
ไม่ว่าผู้จะใช้จะตั้งค่าอะไรมา ระบบจะบีบให้อยู่ใน 9-21 หน่วยกิตเสมอตามระเบียบ

6 **ตรวจแผนที่ได้อีกรอบด้วยกฎชุดเดียวกัน**
ตัวจัดตารางกับตัวตรวจแยกกัน ตัวจัดเสร็จแล้วต้องส่งให้ตัวตรวจอีกที จะได้ไม่มีทางส่งแผนที่ตัวเองตรวจไม่ผ่านให้ผู้ใช้

7 **วาดออกมาให้เห็น พร้อมบอกว่าแก้ยังไง**
ผ่าน = กล่องเขียว / ไม่ผ่าน = กล่องแดงพร้อมรหัสปัญหา และปุ่ม “เปลี่ยนเป็นหมู่ ...” ที่กดแล้วแก้ไขเลยในคลิกเดียว

8 **จำแผนไว้ในเครื่องผู้ใช้**
เก็บใน localStorage ของเบราว์เซอร์ ปิดแล้วเปิดใหม่แผนยังอยู่ และไม่มีข้อมูลถูกส่งออกไปไหน

อีกสองเส้นทางที่เหลือ

ผู้ใช้ทำอะไร	ระบบทำอะไรต่อ	ได้อะไรกลับมา
กดเลือกวิชาเอง จากรายการด้านข้าง	ส่งรายการหมู่เรียนทั้งหมดไปตรวจใหม่ทุกครั้งที่เกิด (ตรวจทุกคู่ว่าเวลาทับกันไหม)	ตารางอัปเดตทันที วิชาที่ชนขึ้นกรอบแดง พร้อมบอกว่าชนกับตัวไหน วันไหน เวลาไหน
ถามเรื่องระเบียบ เช่น “ถอนวิชาได้ถึงเมื่อไหร่”	ค้นเอกสารระเบียบด้วย BM25 + TF-IDF ก่อน แล้วค่อยให้โมเดลภาษาเรียบเรียงจากสิ่งที่ค้นเจอเท่านั้น	คำตอบ + รายการเอกสารที่อ้างอิง ถ้าไม่มีข้อมูลจะบอกตรง ๆ ว่าไม่รู้ไม่เดาให้

06 · สถานะตอนนี้

ใครทำอะไร ถึงไหนแล้ว

ตัวเลขไฟล์นับจากโค้ดจริงบน main ไม่ใช่จากที่แจ้งไว้ — ทุกโมดูลถูกรวมเข้า main เรียบร้อยแล้ว

โมดูล	เจ้าของ	อยู่ที่ branch	ไฟล์โค้ด	สถานะ
01_web_app	เฟิร์ส (First)	main	30	รวมแล้ว
02_api_backend	fffalafair	main	100	รวมแล้ว
03_ai_router_agent	Automatic28m (Phanlop)	main	10	รวมแล้ว
04_course_data_services	— ยังไม่มีเจ้าของ	main	7	มี parser แล้ว
05_data_integration	DevnameJay	main	14	รวมแล้ว
06_schedule_conflict_engine	cheewakorn	main	25	รวมแล้ว
07_rag_llm_engine	เฟิร์ส (First)	main	18	เสร็จแล้ว
08_recommendation_feedback	marisa46054 (Marisa)	main	23	รวมแล้ว
09_main_app	เฟิร์ส (First)	main	29	ขึ้นเว็บแล้ว

 **รวมงานของทุกคนเข้า main แล้ว**

ทั้ง 9 โมดูลอยู่บน main เรียบร้อย ยกทั้งระบบขึ้นพร้อมกันได้ด้วย `docker compose up -d --build` แล้วเปิด `localhost:3000`

ตอนรวม ของใหม่ของแต่ละโมดูลกระจายอยู่คนละสาขา จึงต้องเรียงลำดับให้ของใหม่สุดมาทีหลัง — โมดูล 02 เอาจาก `feat/api-backend`, 03 เอาจาก `develop`, 08 เอาจาก `feat/feedback` — แล้วตรวจยืนยันว่าโค้ดของทุกคนตรงกับสาขาต้นทางทุกไฟล์ ไม่มีใครถูกทับ

07 · คำถามที่ถูกถามบ่อย

ทำไมต้องมีโมดูล 09 ในเมื่อมี 01-08 อยู่แล้ว

เพราะ 01-08 ออกแบบมาเป็น 8 บริการแยกกัน ต้องรัน Docker ทั้งชุดถึงจะใช้ได้ ซึ่งเหมาะกับตอนพัฒนา แต่เอาขึ้นเว็บให้คนทั่วไปใช้ฟรีไม่ได้

เรื่อง	01-08 (ระบบเต็ม)	09 (แอปตัวจริง)
รันยังไง	Docker Compose 8 บริการ + ฐานข้อมูล	เปิดเว็บก็ใช้ได้เลย
จัดตารางด้วย	CP-SAT (OR-Tools) หาคำตอบดีที่สุด	greedy เพราะ OR-Tools หน้า 180 MB เกินโควตา
ตัดคำไทยด้วย	pythainlp	character n-gram เพราะ pythainlp หน้า 250 MB
เหมาะกับ	พัฒนาต่อ แยกกันทำ ขยายทีหลัง	ให้คนใช้จริงวันนี้ โดยไม่มีค่าใช้จ่าย

ทั้งสองฝั่งใช้ รหัสปัญหาชุดเดียวกัน (C1–C6, W1–W5) และข้อความเดียวกัน ออกแบบไว้แบบนี้เพื่อให้วันหลังสลับ 09 ไปเรียกโมดูล 06 ตัวจริงผ่าน HTTP ได้ทันที โดยไม่ต้องแก้หน้าเว็บเลย

[เปิดตัวระบบจริง](#) → [สไลด์บทที่ 7](#) → [เปิดใช้ได้เลย ไม่ต้องสมัครสมาชิก](#)

เอกสารนี้สร้างจากโค้ดจริงใน repo Onpreyaq5/Nuthaluek-CPE-LLM- — จำนวนไฟล์ พอร์ต ชื่อ branch และรหัสปัญหาทั้งหมด ตรวจสอบจากตัวโค้ดโดยตรง ไม่ได้คัดจากเอกสารเก่า